Transformer Network (Part 1)

Introduction to Sequential Data Processing

Kyuhong Shim

Department of Intelligent Software
Sungkyunkwan University

2026. 4. 1.

From Fixed Filters to Adaptive Aggregation

- CNN and FIR filters depend only on recent inputs, and the output is a linear combination of recent inputs:

$$y_t = \sum_{k=0}^{K-1} w_k \, x_{t-k}$$

- **Transformer** generalizes this idea in two ways:
  1. It can aggregate information from the **entire input history**.
  2. The weights are computed **adaptively** from the input itself.
- Thus, Transformer can be viewed as an **adaptive filter** with input-dependent coefficients and unlimited window length.

Adaptive Weighted Sum

- Let each input be a feature vector (i.e., multivariate input). At time t, we want to combine all previous inputs using adaptive weights:

$$\boldsymbol{y}_t = \sum_{k=0}^{t} \alpha_{t,k} \boldsymbol{x}_k, \qquad \boldsymbol{x}_k \in \mathbb{R}^d$$

  - $\alpha_{t,k}$ is an adaptive weight for $\boldsymbol{x}_k$ at time t.
  - $\boldsymbol{y}_t$ is a weighted sum (linear combination) of all inputs.

- To avoid overflow, add constraints to adaptive weights; each weight is positive, and the sum should be 1.

$$\sum_{k=0}^{t} \alpha_{t,k} = 1 \quad \text{and} \quad \alpha_{t,k} > 0.$$

Dot-Product as a Score Function

- The main question is: **how should we compute the weights** $\alpha_{t,k}$?
- The simplest **similarity** measure between two vectors is the dot product.

$$\boldsymbol{a}^\top \boldsymbol{b} = \sum_{i=0}^{d} a_i\, b_i \in \mathbb{R}, \qquad \boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^d$$

  - Recap that $\boldsymbol{a}^\top \boldsymbol{b} = \|\boldsymbol{a}\|\, \|\boldsymbol{b}\| \cos\theta$.

- We first define a **score** between the current timestep t and a previous timestep k:

$$s_{t,k} = \boldsymbol{x}_t^\top \boldsymbol{x}_k$$

  - A larger score means that $\boldsymbol{x}_k$ is more relevant to the current input $\boldsymbol{x}_t$.
  - A smaller score means that $\boldsymbol{x}_k$ is less relevant to the current input $\boldsymbol{x}_t$.

From Scores to Attention Weights

- To satisfy the condition, convert the scores into normalized weights using **softmax**:

$$\alpha_{t,k} = \frac{\exp(s_{t,k})}{\sum_{j=0}^{t} \exp(s_{t,j})}$$

- In summary:

$$\boldsymbol{y}_t = \sum_{k=0}^{t} \alpha_{t,k} \boldsymbol{x}_k = \sum_{k=0}^{t} \underbrace{\left(\frac{\exp(s_{t,k})}{\sum_{j=0}^{t} \exp(s_{t,j})} \right)}_{\boldsymbol{Z}_t} \boldsymbol{x}_k = \sum_{k=0}^{t} \frac{\exp(s_{t,k})}{\boldsymbol{Z}_t} \boldsymbol{x}_k$$

  - This is still a weighted sum, but now the coefficients depend on the input.

Separating the Roles

- The above design uses the same vector $\boldsymbol{x}_t$ for both:
  - **matching**: deciding how much *attention* to pay for each timestep
  - **aggregation**: deciding what information to carry
- This is restrictive because the same input must play two different roles.
- Ideally, we should separate
  - what the current timestep is looking for
  - what each past timestep offers for matching
  - what information is actually passed forward

Introducing Query, Key, and Value

- Introduce three trainable parameters for projections:

$$\boldsymbol{q}_t = \boldsymbol{W}_Q \boldsymbol{x}_t, \qquad \boldsymbol{k}_t = \boldsymbol{W}_K \boldsymbol{x}_t, \qquad \boldsymbol{v}_t = \boldsymbol{W}_V \boldsymbol{x}_t, \qquad \boldsymbol{W}_{Q,K,V} \in \mathbb{R}^{d \times d}$$

  - The output dimensions of $\boldsymbol{q}, \boldsymbol{k}, \boldsymbol{v}$ do not need to be the same as the input, but for now we keep the notation for simplicity.

- We name them **Query**, **Key**, and **Value**:
  - **Query** $\boldsymbol{q}_t$: what the current timestep is looking for
  - **Key** $\boldsymbol{k}_k$: what each timestep offers for matching
  - **Value** $\boldsymbol{v}_k$: the actual content to be aggregated

- The score is now computed by the dot product between the query and the key:

$$s_{t,k} = \boldsymbol{q}_t^\top \boldsymbol{k}_k$$

Scaled Dot Product

- The dot-product score is a sum over d dimensions:

$$\boldsymbol{q}_t^\top \boldsymbol{k}_k = \sum_{i=1}^{d} q_{t,i} k_{k,i}$$

  - As the key dimension d_k grows, the variance of this score also grows.
  - Then softmax can become overly sharp, which may lead to very small gradients.

- To stabilize training, we scale the score by $\sqrt{d}$:

$$s_{t,k} = \frac{\boldsymbol{q}_t^\top \boldsymbol{k}_k}{\sqrt{d}} \qquad \text{(scaled dot-product (SDP))}$$

Example

If $\boldsymbol{q}_t$ and $\boldsymbol{k}_k$ follow $\mathcal{N}(0, \boldsymbol{I})$, then the scaled dot-product output also follows $\mathcal{N}(0, 1)$.

Scaled Dot-Product Attention

- Now we obtain the **scaled dot-product attention (SDPA)** for timestep t:

$$s_{t,k} = \frac{\boldsymbol{q}_t^\top \boldsymbol{k}_k}{\sqrt{d}}$$

$$\alpha_{t,k} = \frac{\exp(s_{t,k})}{\sum_{j=0}^{t} \exp(s_{t,j})}$$

$$\boldsymbol{y}_t = \sum_{k=0}^{t} \alpha_{t,k} \boldsymbol{v}_k$$

- The current input **attends** to previous inputs by *query-key* similarity and aggregates their *values* using adaptive weights.
- Because $\boldsymbol{q}_t, \boldsymbol{k}_k, \boldsymbol{v}_k$ are all computed from the input, this is called **Self-Attention**.

Causal Self-Attention

- Extend this to the causal setting:
  - Timestep t can observe only $0, 1, \cdots, t$ (includes self).
  - The receptive field covers the entire **past** sequence.
- Unlike causal convolution, there is no fixed filter length K.
- Instead, the effective weights are recomputed adaptively for every input.
  - Computation at larger t would be heavier than the computation at smaller t.

- Note that every $\boldsymbol{y}_t$ can be computed in parallel if $\boldsymbol{q}, \boldsymbol{k}, \boldsymbol{v}$ are ready.
  - Therefore, we stack all query, key, and value vectors together, form a matrix, and perform the same computation in parallel.
- Stack all timestep vectors into matrices:

$$\boldsymbol{Q} = \begin{bmatrix} \boldsymbol{q}_0^\top \\ \boldsymbol{q}_1^\top \\ \vdots \\ \boldsymbol{q}_{T-1}^\top \end{bmatrix}, \qquad \boldsymbol{K} = \begin{bmatrix} \boldsymbol{k}_0^\top \\ \boldsymbol{k}_1^\top \\ \vdots \\ \boldsymbol{k}_{T-1}^\top \end{bmatrix}, \qquad \boldsymbol{V} = \begin{bmatrix} \boldsymbol{v}_0^\top \\ \boldsymbol{v}_1^\top \\ \vdots \\ \boldsymbol{v}_{T-1}^\top \end{bmatrix}, \qquad \boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{T \times d}$$

SDPA in Matrix Form

- Equivalently, starting from the input matrix:

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}_0^\top \\ \boldsymbol{x}_1^\top \\ \vdots \\ \boldsymbol{x}_{T-1}^\top \end{bmatrix} \in \mathbb{R}^{T \times d}, \qquad \boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q, \quad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K, \quad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V$$

- Then, SDPA can be written compactly as:

$$\mathrm{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}} + \boldsymbol{M}\right)\boldsymbol{V}$$

  - $\boldsymbol{Q}\boldsymbol{K}^\top \in \mathbb{R}^{T \times T}$ and $\boldsymbol{M} \in \mathbb{R}^{T \times T}$.
  - PyTorch provides a fused operation for SDPA, based on `FlashAttention`.

Common Attention Masking Patterns

- The mask matrix $\boldsymbol{M}$ determines which positions are visible.
- For **causal** self-attention,

$$M_{t,k} = \begin{cases} 0, & k \le t \\ -\infty, & k > t \end{cases}$$

- For **non-causal** self-attention,

$$M_{t,k} = 0 \qquad \text{for all} \quad t, k.$$

- Generally, the mask can enforce many visibility patterns, such as future-looking or local-window attention.

Example: Causal Mask

Example

Below, ✓ means "visible" and × means "masked."
Causal attention sees only the past.

Causal mask

	0	1	2	3	4	5	6	7
0	✓	×	×	×	×	×	×	×
1	✓	✓	×	×	×	×	×	×
2	✓	✓	✓	×	×	×	×	×
3	✓	✓	✓	✓	×	×	×	×
4	✓	✓	✓	✓	✓	×	×	×
5	✓	✓	✓	✓	✓	✓	×	×
6	✓	✓	✓	✓	✓	✓	✓	×
7	✓	✓	✓	✓	✓	✓	✓	✓

SUNG KYUN KWAN UNIVERSITY(SKKU)

Example

Look-ahead attention sees only the current and a few future positions.

Look-ahead mask

	0	1	2	3	4	5	6	7
0	✓	✓	×	×	×	×	×	×
1	✓	✓	✓	×	×	×	×	×
2	✓	✓	✓	✓	×	×	×	×
3	✓	✓	✓	✓	✓	×	×	×
4	✓	✓	✓	✓	✓	✓	×	×
5	✓	✓	✓	✓	✓	✓	✓	×
6	✓	✓	✓	✓	✓	✓	✓	✓
7	✓	✓	✓	✓	✓	✓	✓	✓

SUNG KYUN KWAN UNIVERSITY(SKKU)

Example

Local-window attention restricts attention to nearby positions.
This enforces locality, similar to convolution.

Local-window mask (radius 1)

	0	1	2	3	4	5	6	7
0	✓	✓	×	×	×	×	×	×
1	✓	✓	✓	×	×	×	×	×
2	×	✓	✓	✓	×	×	×	×
3	×	×	✓	✓	✓	×	×	×
4	×	×	×	✓	✓	✓	×	×
5	×	×	×	×	✓	✓	✓	×
6	×	×	×	×	×	✓	✓	✓
7	×	×	×	×	×	×	✓	✓

- A single attention operation may be insufficient to capture all types of relationships.
- **Multi-Head Self-Attention (MHSA)** runs several attention modules in parallel:

$$\text{head}_h = \text{Attention}\left(\boldsymbol{Q}^{(h)}, \boldsymbol{K}^{(h)}, \boldsymbol{V}^{(h)}\right)$$

- Each head has its own trainable parameters:

$$\boldsymbol{Q}^{(h)} = \boldsymbol{X}\boldsymbol{W}_Q^{(h)}, \quad \boldsymbol{K}^{(h)} = \boldsymbol{X}\boldsymbol{W}_K^{(h)}, \quad \boldsymbol{V}^{(h)} = \boldsymbol{X}\boldsymbol{W}_V^{(h)}, \qquad \boldsymbol{Q}^{(h)}, \boldsymbol{K}^{(h)}, \boldsymbol{V}^{(h)} \in \mathbb{R}^{T \times d_h}$$

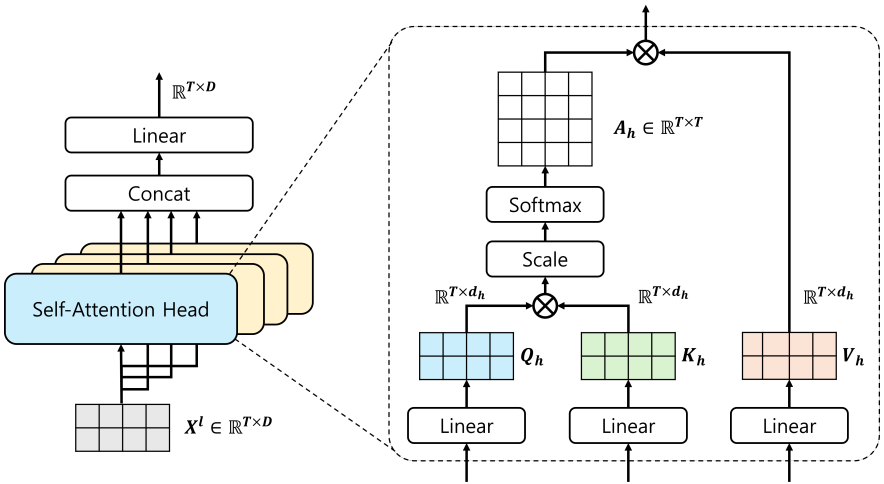
  - Let the number of heads be H, then we often set $d = H \times d_h$.
  - This design does not increase the number of parameters as H increases.

- Finally, the outputs are concatenated and projected via $\boldsymbol{W}_O$:

$$\mathrm{MultiHead}(\boldsymbol{X}) = \mathrm{Concat}(\mathrm{head}_1, \cdots, \mathrm{head}_H)\,\boldsymbol{W}_O$$

  - The output projection mixes the results from every attention head.
  - Note that there is no *nonlinear* activation function inside MHSA.

From Single-Head to Multi-Head Attention



Positional Encoding

- SA compares inputs by content, but it does not explicitly encode their order.
- Without positional information, the model cannot distinguish sequences that contain the same inputs in different orders.
  - $\{A, B, C, D, A\}$ vs. $\{A, C, D, B, A\}$ vs. $\{C, B, D, A, A\}$.
- We need a method to encode positional information; **Positional Encoding**.
- To address this problem, we can add a position-dependent vector to each input:

$$\tilde{\boldsymbol{x}}_t = \boldsymbol{x}_t + \boldsymbol{p}_t$$

- Then the attention mechanism operates on the position-aware representation:

$$\boldsymbol{q}_t = \boldsymbol{W}_Q \tilde{\boldsymbol{x}}_t, \quad \boldsymbol{k}_t = \boldsymbol{W}_K \tilde{\boldsymbol{x}}_t, \quad \boldsymbol{v}_t = \boldsymbol{W}_V \tilde{\boldsymbol{x}}_t$$

Absolute Positional Embeddings

- For the additive positional encoding, $\boldsymbol{p}_t$ depends on the **absolute position** t.
- This type of PE is called **Absolute Positional Embedding (APE)**.
  - (1) Trainable APEs: $\boldsymbol{p}_t$ as parameters.
  - (2) Sinusoidal APEs: $\boldsymbol{p}_t = f(t)$, computed by the rule-based function.
- Limitations of APEs:
  - They do not directly encode relative distance ($t \leftrightarrow t + k$ vs. $t + 10 \leftrightarrow t + k + 10$).
  - Learned embeddings may not generalize well to longer unseen sequences.
  - In time series, relative offsets are often more important than absolute indices.

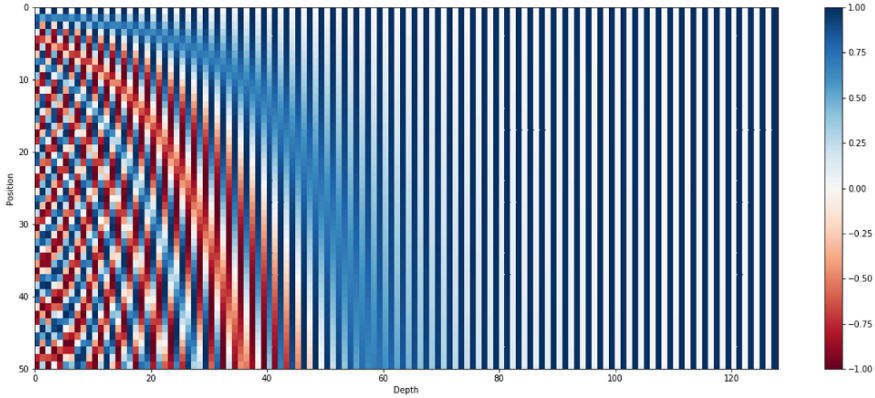
- **Sinusoidal positional encoding** defines each position using sinusoids (i.e., sine and cosine functions) of different frequencies:

$$p_t^{(2i)} = \sin\left(\frac{t}{10000^{2i/d}}\right) \qquad \text{(even elements)}$$

$$p_t^{(2i+1)} = \cos\left(\frac{t}{10000^{2i/d}}\right) \qquad \text{(odd elements)}$$

- Intuition:
  - Different dimensions correspond to different frequencies, from high to low.
  - Smaller $i \rightarrow$ faster oscillation $\rightarrow$ higher frequency.
  - Nearby positions receive similar encodings, while distant positions become more distinguishable.

Sinusoidal Positional Encoding



- Unlike learned absolute embeddings, sinusoidal encodings can be extended to longer sequence lengths without introducing new parameters.

Example: Changing the Base in Sinusoidal Encoding

SUNG KYUN KWAN UNIVERSITY(SKKU)

Example

What does it mean if we replace the base 10000 with 100000 in

$$p_t^{(2i)} = \sin\left(\frac{t}{10000^{2i/d}}\right), \qquad p_t^{(2i+1)} = \cos\left(\frac{t}{10000^{2i/d}}\right)?$$

- Replacing 10000 with 100000 makes the encoding vary more slowly with position.
  - Larger base $\rightarrow$ longer wavelengths $\rightarrow$ slower variation across time.
  - It increases the base of the geometric frequency scale, so the frequencies become more spread out toward lower frequencies.
  - PE changes more slowly across positions, especially in higher dimensions.
  - More suitable to capture long-range positional structure.

- The attention matrix is often called **Attention Weights**:

$$\boldsymbol{A} = \mathrm{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}} + \boldsymbol{M}\right)$$

  - Different layers and different heads show different attention weights.
  - By inspecting $\boldsymbol{A}$, we can see which timesteps receive large weights.

- This often gives a more interpretable view of information flow.
- However, attention weights should still be interpreted with care; they are *not always a complete explanation* of the model's decision.

Example

What does it mean if the attention weights are strongly concentrated near the diagonal?

Computational Cost of Self-Attention

- For a sequence of length T, each attention head forms a score matrix

$$\boldsymbol{Q}^{(h)}\boldsymbol{K}^{(h)\top} \in \mathbb{R}^{T\times T}$$

- If there are H heads and each head has key dimension d_k, then computation cost for the attention score scales as $\mathcal{O}(HT^2d_h) = \mathcal{O}(T^2d)$.
- Both computation and memory grow quadratically with the sequence length T, and grow linearly with the number of attention heads H.

Computational Cost of Self-Attention

- We also need the linear projections for Q, K, V, and O:

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q, \qquad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K, \qquad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V$$

- Each of their computation costs is $\mathcal{O}(Td^2)$
- The total cost of forming $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ is $\mathcal{O}(3Td^2)$.
- Including the final output projection, the cost becomes $\mathcal{O}(4Td^2)$.

Example

What happens if d is much larger than T? This is the case for modern LLMs.

Advantages and Limitations of Self-Attention

- **Advantages**
  - Adaptive, input-dependent weighting
  - Direct access to long-range dependencies
  - Highly parallelizable computation
  - Flexible receptive field over the full history
- **Limitations**
  - Quadratic cost in sequence length
  - No built-in notion of order without positional encoding
  - May require more data and computation than simpler models